
```
# Get a resized version of the base image:
magentaWidth = magentaRight - magentaLeft + 1
magentaHeight = magentaBottom - magentaTop + 1
baseImageAspectRatio = baseImageWidth / baseImageHeight
magentaAspectRatio = magentaWidth / magentaHeight
```

From `magentaRight` and `magentaLeft`, we can calculate the width of the magenta area. The `+ 1` accounts for a small, necessary adjustment: if the right side of the magenta area was the x-coordinate of 11 and the left side was 10, the width would be two pixels. This is correctly calculated by `(magentaRight - magentaLeft + 1)`, not `(magentaRight - magentaLeft)`.

Because the aspect ratio is the width divided by the height, images with large aspect ratios are taller than they are wide, and those with small aspect ratios are wider than they are tall. An aspect ratio of 1.0 describes a perfect square. The next lines set the dimensions of the resized image after comparing the aspect ratios of the base image and the magenta area:

```
if baseImageAspectRatio < magentaAspectRatio:
    # Make the resized width match the width of the magenta area:
    widthRatio = magentaWidth / baseImageWidth
    resizedImage = baseImage.resize((magentaWidth,
    int(baseImageHeight * widthRatio) + 1), Image.NEAREST)
else:
    # Make the resized height match the height of the magenta area:
    heightRatio = magentaHeight / baseImageHeight
    resizedImage = baseImage.resize((int(baseImageWidth *
    heightRatio) + 1, magentaHeight), Image.NEAREST)
```

If the base image's aspect ratio is less than the magenta area's aspect ratio, the resized image's width should match the width of the magenta area. If the base image's aspect ratio is greater, the resized image's height should match the height of the magenta area. We then determine the other dimension by multiplying the base image's height by the width ratio, or the base image's width by the height ratio. This ensures that the resized image both completely covers the magenta area and remains proportional to its original aspect ratio.

We call the `resize()` method once to produce a new Pillow `Image` object resized to match either the width of the base image or the height of the base image. The first argument is a (width, height) tuple for the new image's size.